

Entregable 1: Introducción y entendimiento del problema.

Rafael Emilio Funez Perdomo A00841342
Alan Robles Gracia A00841816
Emma Huerta Villafuerte A01723584
Sebastián Alemán Cervantes A00841034

Curso:
Diseño de algoritmos matemáticos bioinspirados (Gpo 201)

Profesor:
Fernando Elizalde Ramírez

Campus: Monterrey

13 de octubre de 2026

Índice

1. Introducción **3**
1.1. El problema de ruteo de vehículos con capacidad limitada (CVRP) . . . 3
1.2. Costos logísticos asociados al ruteo de vehículos . . . 3
1.3. Impacto al medio ambiente . . . 4
1.4. Objetivo de la situación problema . . . 4
1.5. Contexto: logística urbana de última milla . . . 5

2. Trabajo relacionado **5**

3. Solución manual de la instancia CVRP_10 **7**
3.1. Descripción de la instancia . . . 7
3.2. Matriz de costos . . . 8
3.3. Método de solución: heurística de ahorros . . . 9
3.4. Construcción de las rutas . . . 10
3.5. Resultados . . . 10
3.6. Cota inferior mediante el TSP y representación de gran ruta . . . 12

4. Definición del problema **13**

5. Modelo matemático **13**
5.1. Diagrama del modelo . . . 14

6. Solución con el modelo exacto **15**
6.1. Efecto del número de vehículos . . . 16

7. Algoritmo de solución en Python **17**

8. Comparación de resultados **17**

9. Conclusiones **19**

Referencias **20**

A. Registro completo de la heurística de ahorros **22**

B. Repositorio de código y estructura del proyecto **23**
B.1. Estructura de carpetas . . . 23
B.2. Módulos del directorio `src/` . . . 24
B.3. Scripts de ejecución y reproducción . . . 24

1. Introducción

La distribución de bienes desde un centro de abastecimiento hacia un conjunto de clientes geográficamente dispersos es una de las actividades centrales de cualquier cadena de suministro. Decidir qué vehículo atiende a qué clientes y en qué orden los visita determina, en buena medida, el costo operativo, el nivel de servicio y la huella ambiental de la operación logística. Este tipo de decisiones se estudia en la investigación de operaciones bajo el nombre genérico de *Problema de Ruteo de Vehículos* (*Vehicle Routing Problem*, VRP), formulado por primera vez por Dantzig y Ramser Dantzig y Ramser, 1959 al modelar la distribución de combustible a estaciones de servicio.

1.1. El problema de ruteo de vehículos con capacidad limitada (CVRP)

El *Capacitated Vehicle Routing Problem* (CVRP) es la versión clásica y más estudiada de la familia VRP Laporte, 2009; Toth y Vigo, 2014. En él se cuenta con un depósito central, una flota homogénea de vehículos con capacidad máxima Q y un conjunto de n clientes, cada uno con una demanda conocida q_i y una ubicación determinada. El objetivo es diseñar un conjunto de rutas de costo mínimo tales que:

- cada ruta inicia y termina en el depósito;
- cada cliente es visitado exactamente una vez por un único vehículo;
- la suma de las demandas de los clientes atendidos en una ruta no excede la capacidad Q del vehículo.

El problema puede representarse sobre un grafo $G = (V, A)$, donde $V = \{0, 1, \ldots, n\}$ es el conjunto de nodos (el nodo 0 corresponde al depósito) y cada arco $(i, j) \in A$ tiene asociado un costo c_{ij}, usualmente la distancia o el tiempo de viaje entre los nodos i y j. Cuando los nodos están definidos por coordenadas (x, y), el costo suele calcularse como la distancia euclidiana entre ellos.

El CVRP generaliza al Problema del Agente Viajero (TSP), por lo que pertenece a la clase de problemas NP-difíciles Lenstra y Rinnooy Kan, 1981. En la práctica, esto significa que el esfuerzo computacional de los métodos exactos crece exponencialmente con el número de clientes: instancias pequeñas pueden resolverse a optimalidad con programación entera mixta, pero instancias de tamaño realista requieren heurísticas y metaheurísticas, entre ellas los algoritmos bioinspirados (algoritmos genéticos, colonias de hormigas, enjambres de partículas, etc.), que buscan soluciones de alta calidad en tiempos razonables.

1.2. Costos logísticos asociados al ruteo de vehículos

El transporte representa uno de los componentes de mayor peso dentro del costo logístico total de una empresa. Los costos atribuibles directamente a las decisiones de ruteo pueden clasificarse en:

- **Costos variables por distancia:** combustible, desgaste de llantas, mantenimiento y peajes, proporcionales a los kilómetros recorridos.
- **Costos variables por tiempo:** salarios del personal de reparto, horas extra y costos de oportunidad por tiempo de ciclo.

- **Costos fijos por vehículo:** adquisición o arrendamiento, seguros, permisos y depreciación; cada ruta adicional implica, en general, un vehículo o un turno adicional.
- **Costos por subutilización de capacidad:** un vehículo que sale con una fracción importante de su capacidad vacía incurre prácticamente en el mismo costo de traslado que uno lleno, lo que eleva el costo por unidad entregada.

Debido a esta estructura, una mala planeación de rutas se traduce en recorridos más largos, en un número de vehículos mayor al necesario y en espacio de carga desperdiciado. La literatura reporta que el uso de herramientas computacionales de planeación de rutas puede generar ahorros del orden de 5 % a 20 % en los costos de transporte Toth y Vigo, 2014, lo que justifica el interés práctico y académico en el CVRP.

1.3. Impacto al medio ambiente

El transporte de carga por carretera depende mayoritariamente de combustibles fósiles y es una fuente relevante de emisiones de dióxido de carbono (CO_2), óxidos de nitrógeno (NO_x) y partículas suspendidas, además de contribuir a la congestión vial y al ruido urbano. Dado que el consumo de combustible está estrechamente ligado a la distancia recorrida y a la carga transportada, el ruteo de vehículos tiene un efecto directo sobre la huella ambiental de la distribución.

Reducir la distancia total recorrida y el número de vehículos en circulación disminuye proporcionalmente el consumo de combustible y las emisiones asociadas. Esta relación ha motivado variantes del problema, como el *Green VRP* y el *Pollution-Routing Problem* Bektaş y Laporte, 2011, que incorporan explícitamente las emisiones en la función objetivo. Aun en su forma clásica, minimizar la distancia del CVRP constituye una estrategia directa de mitigación ambiental, alineada con los objetivos de sostenibilidad que hoy se exigen a las cadenas de suministro.

1.4. Objetivo de la situación problema

El objetivo de esta situación problema es diseñar el conjunto de rutas que permita atender la demanda de todos los clientes desde un depósito central, **minimizando la distancia total recorrida** por la flota y respetando la capacidad de cada vehículo, de modo que se reduzcan tanto los costos logísticos como el impacto ambiental de la operación.

Para ello, en esta primera etapa se plantean los siguientes objetivos específicos:

1. Caracterizar el CVRP y revisar trabajos relacionados que aborden su solución mediante métodos exactos, heurísticos y bioinspirados.
2. Resolver manualmente la instancia `CVRP_10`, construyendo la matriz de costos a partir de las coordenadas (x, y) de los nodos y reportando la función objetivo, el número de rutas, el porcentaje de capacidad desperdiciada por ruta y el trazo de las rutas.
3. Formular un modelo matemático de programación entera mixta para el CVRP y resolver la instancia con un optimizador.

4. Programar un algoritmo de solución y comparar, en términos porcentuales (GAP), el costo de su solución y su tiempo de cómputo frente al modelo exacto.

1.5. Contexto: logística urbana de última milla

El caso de estudio describe una operación de entregas urbanas con varias decisiones que pueden formularse como problemas de optimización combinatoria (Tabla 1). El caso en sí es material del curso; si el profesor proporcionó la fuente periodística original, debe citarse aquí. Un algoritmo bioinspirado es adecuado cuando el problema es NP-difícil, su espacio de soluciones crece de forma combinatoria y el tamaño real supera lo que los métodos exactos resuelven en tiempo razonable (Blum & Roli, 2003). No lo es cuando existe un algoritmo polinomial o exacto eficiente, como el camino más corto entre dos puntos (Dijkstra, 1959). Las asignaciones de algoritmo de la tabla se apoyan directamente en la literatura para el CVRP (Ai & Kachitvichyanukul, 2009; Bullnheimer et al., 1999; Prins, 2004; Reimann et al., 2004); para el resto de las filas son una propuesta nuestra.

Tabla 1: Situaciones del caso modeladas como problemas de optimización.

Situación del caso	Problema de optimización	¿Bioinspirado?	Algoritmo sugerido
Asignar y ordenar paradas (p. ej., 16 por hora)	CVRP (Toth & Vigo, 2014)	Sí: NP-difícil y de gran tamaño	GA con *split* (Prins, 2004); ACO (Reimann et al., 2004)
Horarios y prohibiciones de entrega	VRP con ventanas de tiempo (Solomon, 1987)	Sí, con manejo de infactibilidad	GA híbrido (Vidal, 2022); ACO
Tiempo de estacionamiento y entregas fallidas	VRP estocástico (Gendreau et al., 1996)	Sí, evaluando por escenarios	GA o ACO con simulación
Centros logísticos urbanos	Localización–ruteo (Prodhon & Prins, 2014) y ruteo en dos niveles (Crainic et al., 2009)	Sí por el tamaño conjunto	GA con codificación de apertura y ruteo
Casilleros y recolección en tienda	Localización de instalaciones (p-mediana)	Parcial: exacto en tamaños moderados	GA o PSO discreto
eBikes, flotas eléctricas y emisiones	VRP eléctrico (Schneider et al., 2014) y Pollution-Routing (Bektaş & Laporte, 2011)	Sí, es multiobjetivo	GA multiobjetivo (NSGA-II, Deb et al., 2002)
Drones y robots de entrega	Ruteo camión–dron (Murray & Chu, 2015)	Sí, por el acoplamiento	GA o ACO adaptados
Camino más corto entre dos puntos	Ruta más corta	No: es polinomial (Dijkstra, 1959)	Dijkstra

2. Trabajo relacionado

Desde su formulación por Dantzig y Ramser Dantzig y Ramser, 1959, el CVRP ha sido abordado mediante tres grandes familias de métodos: (i) métodos **exactos**, basados en programación entera y técnicas

de ramificación y acotamiento, que garantizan la optimalidad pero cuyo tiempo de cómputo crece rápidamente con el tamaño de la instancia; (ii) **heurísticas constructivas y de mejora**, que generan soluciones factibles de forma rápida aunque sin garantía de optimalidad; y (iii) **metaheurísticas**, entre ellas los algoritmos bioinspirados, que exploran el espacio de soluciones imitando procesos naturales como la evolución, el forrajeo de las hormigas o el comportamiento de enjambres Laporte, 2009; Toth y Vigo, 2014.

A continuación se describen trabajos representativos de cada familia, con énfasis en los enfoques bioinspirados por ser el objeto de estudio de este proyecto.

Heurística de ahorros. Clarke y Wright Clarke y Wright, 1964 propusieron una de las heurísticas constructivas más utilizadas para el CVRP. Parte de una solución en la que cada cliente es atendido por una ruta independiente y fusiona rutas de manera iterativa según el *ahorro* $s_{ij} = c_{0i} + c_{0j} - c_{ij}$, siempre que la capacidad lo permita. Su sencillez la ha convertido en un punto de referencia y en un generador de soluciones iniciales para metaheurísticas.

Métodos exactos. Fukasawa et al. Fukasawa et al., 2006 desarrollaron un algoritmo de ramificación, corte y generación de columnas (*branch-and-cut-and-price*) que combina desigualdades válidas de capacidad con una formulación de rutas. Este enfoque permitió demostrar la optimalidad de diversas instancias clásicas de la literatura que hasta entonces permanecían abiertas, aunque con tiempos de cómputo elevados.

Algoritmos genéticos. Baker y Ayechew Baker y Ayechew, 2003 aplicaron un algoritmo genético al CVRP y mostraron que, por sí solo, no alcanzaba la calidad de las mejores metaheurísticas de su época, pero que al hibridarlo con búsqueda local en vecindarios se volvía competitivo. Prins Prins, 2004 propuso un algoritmo evolutivo híbrido (memético) que codifica cada individuo como una *gran ruta* sin delimitadores y utiliza un procedimiento óptimo de partición (*split*) para dividirla en rutas factibles; este fue uno de los primeros algoritmos genéticos en superar a los métodos de búsqueda tabú en las instancias clásicas.

Colonias de hormigas. Bullnheimer, Hartl y Strauss Bullnheimer et al., 1999 adaptaron el *Ant System* al CVRP, incorporando la información de ahorros en la regla de decisión de las hormigas, listas de candidatos y una estrategia de actualización de feromona basada en las mejores soluciones (*rank-based*), lo que mejoró notablemente el desempeño respecto a la versión original.

Enjambre de partículas. Ai y Kachitvichyanukul Ai y Kachitvichyanukul, 2009 propusieron dos representaciones de solución para resolver el CVRP con optimización por enjambre de partículas (PSO), donde la posición de cada partícula se decodifica en un conjunto de rutas. Sus resultados mostraron que PSO puede obtener soluciones cercanas a las mejores conocidas en instancias de referencia.

Estado del arte. Vidal et al. (Vidal et al., 2012) introdujeron la Hybrid Genetic Search with Adaptive Diversity Control (HGSADC) para problemas multidepósito y periódicos, que combina un algoritmo genético con búsqueda local intensiva, manejo de soluciones infactibles penalizadas y control explícito de la diversidad

poblacional. Para el CVRP, Vidal (2022) presentó HGS-CVRP, una implementación abierta con el vecindario SWAP*, que se considera de los métodos más efectivos para este problema.

La Tabla 2 resume y compara los trabajos revisados.

Tabla 2: Tabla comparativa de trabajos relacionados con el CVRP.

Autor(es) y año	**Método**	**Tipo**	**Aportación principal**	**Ventajas / limitaciones**
Clarke y Wright (1964) Clarke y Wright, 1964	Heurística de ahorros	Heurística constructiva	Fusión iterativa de rutas según el ahorro de distancia.	Muy rápida y fácil de implementar; sin garantía de optimalidad y con poca capacidad de mejora.
Bullnheimer et al. (1999) Bullnheimer et al., 1999	Ant System mejorado	Bioinspirado (ACO)	Uso de ahorros en la visibilidad, listas de candidatos y actualización de feromona por rango.	Construye soluciones de buena calidad; sensible al ajuste de parámetros (α, β, ρ).
Baker y Ayechew (2003) Baker y Ayechew, 2003	Algoritmo genético híbrido	Bioinspirado (GA)	Hibridación del GA con búsqueda en vecindarios.	Muestra que la búsqueda local es clave; el GA puro no fue competitivo.
Prins (2004) Prins, 2004	Algoritmo memético con *split*	Bioinspirado (GA + búsqueda local)	Codificación como gran ruta y partición óptima en rutas factibles.	Representación simple y operadores del TSP reutilizables; alto desempeño en instancias clásicas.
Fukasawa et al. (2006) Fukasawa et al., 2006	Branch-and-cut-and-price	Exacto	Combinación de cortes de capacidad y generación de columnas.	Garantiza optimalidad; tiempo de cómputo elevado en instancias grandes.
Ai y Kachitvichyanukul (2009) Ai y Kachitvichyanukul, 2009	Optimización por enjambre de partículas	Bioinspirado (PSO)	Dos representaciones de solución para decodificar partículas en rutas.	Pocos parámetros y fácil implementación; la calidad depende de la decodificación.
Vidal et al. (2012) Vidal et al., 2012	HGSADC	Bioinspirado (GA híbrido)	Control adaptativo de diversidad y manejo de soluciones infactibles.	Resultados de estado del arte; implementación más compleja.

Del análisis se observa que los métodos exactos son viables para instancias pequeñas o medianas, mientras que los algoritmos bioinspirados destacan cuando se hibridan con procedimientos de búsqueda local. Esto motiva el enfoque del presente proyecto: resolver la instancia `CVRP_10` con un modelo exacto como referencia de optimalidad y comparar contra ella, mediante el GAP porcentual, el costo y el tiempo de cómputo del algoritmo desarrollado.

3. Solución manual de la instancia CVRP_10

3.1. Descripción de la instancia

La instancia `CVRP_10` consta de un depósito y $n = 10$ clientes, con una capacidad por vehículo de $Q = 50$ unidades. Para mantener la notación de la Sección 1, los nodos se re-indexaron de modo que el depósito es el nodo 0 y los clientes son los nodos $1, \ldots, 10$ (el nodo k del archivo original corresponde al nodo $k - 1$). La Tabla 3 muestra las coordenadas y demandas, y la Figura 1 su ubicación.

Tabla 3: Coordenadas y demandas de la instancia CVRP_10.

Nodo	x	y	Demanda q_i	Tipo
0	50	50	0	Depósito
1	20	70	12	Cliente
2	30	80	18	Cliente
3	60	75	10	Cliente
4	80	70	15	Cliente
5	25	45	14	Cliente
6	40	30	20	Cliente
7	65	25	16	Cliente
8	85	40	11	Cliente
9	45	60	13	Cliente
10	70	55	17	Cliente
Total			146	

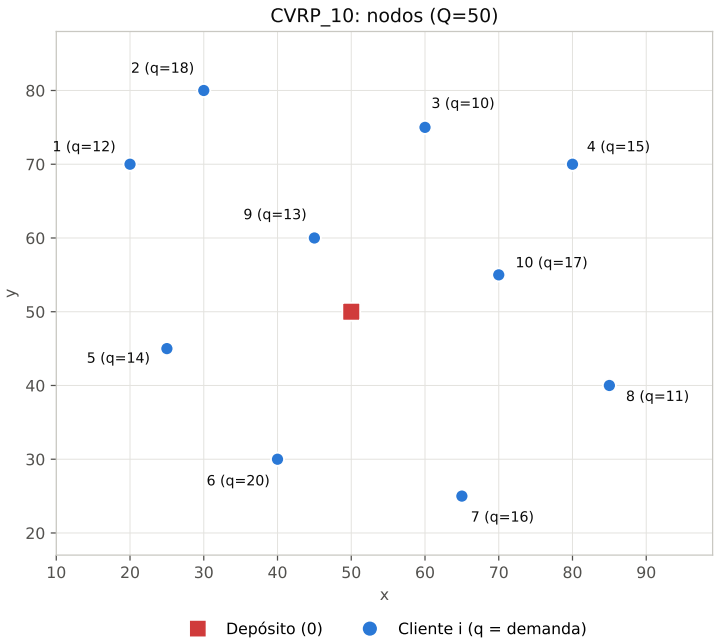


Figura 1: Ubicación del depósito y de los clientes con su demanda.

La demanda total es $\sum_i q_i = 146$, por lo que el número mínimo de vehículos necesarios es

$$K_{\text{mín}} = \left\lceil \frac{146}{50} \right\rceil = 3.$$

3.2. Matriz de costos

El costo de viajar del nodo i al nodo j se definió como la distancia euclidiana:

$$c_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}.$$

La Tabla 4 presenta la matriz resultante. Es simétrica ($c_{ij} = c_{ji}$) y con diagonal cero. El cliente más cercano al depósito es el 9 (11.18) y el más lejano es el 8 (36.40).

Tabla 4: Matriz de costos c_{ij} (distancia euclidiana).

c_{ij}	0	1	2	3	4	5	6	7	8	9	10
0	0.00	36.06	36.06	26.93	36.06	25.50	22.36	29.15	36.40	11.18	20.62
1	36.06	0.00	14.14	40.31	60.00	25.50	44.72	63.64	71.59	26.93	52.20
2	36.06	14.14	0.00	30.41	50.99	35.36	50.99	65.19	68.01	25.00	47.17
3	26.93	40.31	30.41	0.00	20.62	46.10	49.24	50.25	43.01	21.21	22.36
4	36.06	60.00	50.99	20.62	0.00	60.42	56.57	47.43	30.41	36.40	18.03
5	25.50	25.50	35.36	46.10	60.42	0.00	21.21	44.72	60.21	25.00	46.10
6	22.36	44.72	50.99	49.24	56.57	21.21	0.00	25.50	46.10	30.41	39.05
7	29.15	63.64	65.19	50.25	47.43	44.72	25.50	0.00	25.00	40.31	30.41
8	36.40	71.59	68.01	43.01	30.41	60.21	46.10	25.00	0.00	44.72	21.21
9	11.18	26.93	25.00	21.21	36.40	25.00	30.41	40.31	44.72	0.00	25.50
10	20.62	52.20	47.17	22.36	18.03	46.10	39.05	30.41	21.21	25.50	0.00

3.3. Método de solución: heurística de ahorros

Para la solución manual se aplicó la versión paralela de la heurística de ahorros de Clarke y Wright (1964):

1. Se inicia con una ruta independiente por cliente: $0 \to i \to 0$.
2. Para cada par de clientes se calcula el ahorro $s_{ij} = c_{0i} + c_{0j} - c_{ij}$, que es la distancia que se deja de recorrer al atender i y j en la misma ruta.
3. Los ahorros se ordenan de mayor a menor.
4. Se recorre la lista y se fusionan las rutas que contienen a i y a j solo si: (a) están en rutas distintas, (b) ambos son extremos de su ruta (adyacentes al depósito) y (c) la carga combinada no excede Q.

La Tabla 5 muestra los 15 mayores ahorros.

Tabla 5: Los 15 mayores ahorros s_{ij}.

#	(i, j)	c_{0i}	c_{0j}	c_{ij}	s_{ij}
1	$(1, 2)$	36.06	36.06	14.14	57.97
2	$(3, 4)$	26.93	36.06	20.62	42.37
3	$(4, 8)$	36.06	36.40	30.41	42.04
4	$(7, 8)$	29.15	36.40	25.00	40.56
5	$(4, 10)$	36.06	20.62	18.03	38.64
6	$(1, 5)$	36.06	25.50	25.50	36.06
7	$(8, 10)$	36.40	20.62	21.21	35.80
8	$(2, 3)$	36.06	26.93	30.41	32.57
9	$(5, 6)$	25.50	22.36	21.21	26.64
10	$(2, 5)$	36.06	25.50	35.36	26.20
11	$(6, 7)$	22.36	29.15	25.50	26.02
12	$(3, 10)$	26.93	20.62	22.36	25.18
13	$(1, 3)$	36.06	26.93	40.31	22.67
14	$(2, 9)$	36.06	11.18	25.00	22.24
15	$(2, 4)$	36.06	36.06	50.99	21.12

3.4. Construcción de las rutas

De los 45 pares evaluados, solo 6 produjeron una fusión (Tabla 6); el registro completo está en el Anexo A. Partiendo de 10 rutas individuales, 6 fusiones dan como resultado 4 rutas.

Tabla 6: Fusiones aceptadas por la heurística de ahorros.

Iter.	(i, j)	s_{ij}	Carga	Ruta resultante
1	$(1, 2)$	57.97	30/50	0-1-2-0
2	$(3, 4)$	42.37	25/50	0-3-4-0
3	$(4, 8)$	42.04	36/50	0-3-4-8-0
6	$(1, 5)$	36.06	44/50	0-2-1-5-0
11	$(6, 7)$	26.02	36/50	0-6-7-0
20	$(3, 9)$	16.89	49/50	0-8-4-3-9-0

Dos rechazos tempranos definen la solución. En la iteración 4, el par $(7, 8)$ no se fusionó porque la carga combinada ($16 + 36 = 52$) excede Q. En la iteración 5, el par $(4, 10)$ no se fusionó porque el nodo 4 ya era interior de la ruta 0-3-4-8-0. A partir de ahí, ninguna fusión con el cliente 10 fue posible: la mayoría excedía la capacidad y los pares $(4, 10)$ y $(1, 10)$ fallaron porque el nodo ya era interior de su ruta; por ello el cliente 10 quedó atendido en una ruta individual.

3.5. Resultados

La Tabla 7 resume la solución y la Figura 2 traza las rutas. El porcentaje de espacio desperdiciado por ruta k se calculó como

$$\%D_k = \frac{Q - \text{carga}_k}{Q} \times 100.$$

Tabla 7: Solución obtenida con la heurística de ahorros.

Ruta	Secuencia	Carga	Distancia	Espacio desperdiciado (%)
1	0-2-1-5-0	44	101.19	12.00
2	0-8-4-3-9-0	49	119.82	2.00
3	0-6-7-0	36	77.01	28.00
4	0-10-0	17	41.23	66.00
Total	4 rutas	146	339.25	27.00
Capacidad $Q = 50$; demanda total = 146; vehículos mínimos = $\lceil 146/50 \rceil = 3$				

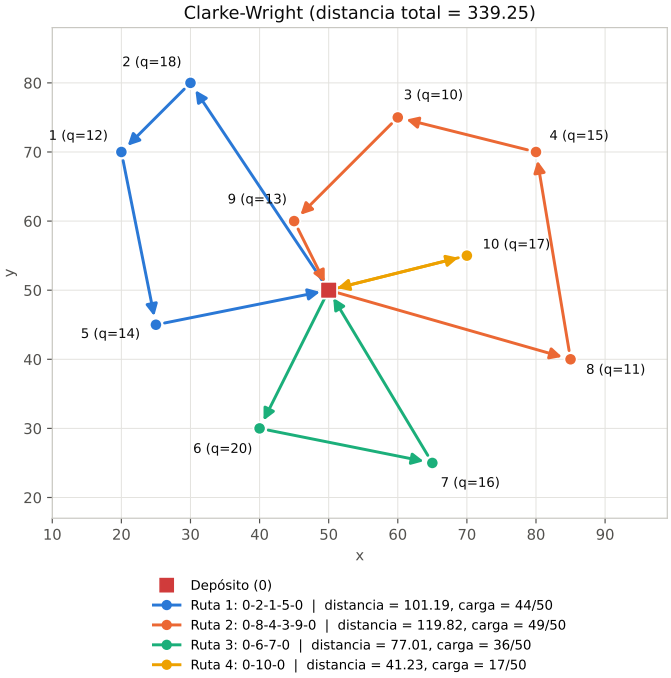


Figura 2: Rutas obtenidas con la heurística de ahorros.

- **Función objetivo:** $Z = 339.25$ unidades de distancia.
- **Número de rutas:** 4.
- **Espacio desperdiciado:** 12 %, 2 %, 28 % y 66 % por ruta; 27 % global, es decir, $(4 \cdot 50 - 146)/(4 \cdot 50)$.

La solución usa una ruta más que el mínimo $K_{\text{mín}} = 3$, y esa cuarta ruta (0-10-0) viaja con 66 % de su capacidad vacía. Con 3 rutas la demanda sí cabe: por ejemplo, $\{6, 2, 1\}$ suma 50, $\{10, 7, 5\}$ suma 47 y $\{4, 8, 9, 3\}$ suma 49. Sin embargo, el objetivo es minimizar la distancia y no el número de vehículos, por lo que usar una ruta extra no implica por sí mismo una mala solución. El modelo exacto de la Sección 6 permite medir qué tan lejos está esta solución del óptimo.

3.6. Cota inferior mediante el TSP y representación de gran ruta

Si se ignora la capacidad Q, el problema se reduce al Problema del Agente Viajero (TSP) sobre los 11 nodos. Como los costos son distancias euclidianas, que cumplen la desigualdad triangular, las rutas de cualquier solución del CVRP pueden encadenarse a través del depósito en un recorrido cerrado de igual longitud, y las visitas repetidas al depósito se eliminan sin aumentarla. Por tanto, el costo del TSP óptimo es una cota inferior del CVRP (Toth & Vigo, 2014). Se resolvió el TSP de forma exacta con el algoritmo de programación dinámica de Held y Karp (Held & Karp, 1962), obteniendo $Z_{\text{TSP}} = 234.31$ con el recorrido 0-3-4-10-8-7-6-5-1-2-9-0 (Figura 3).

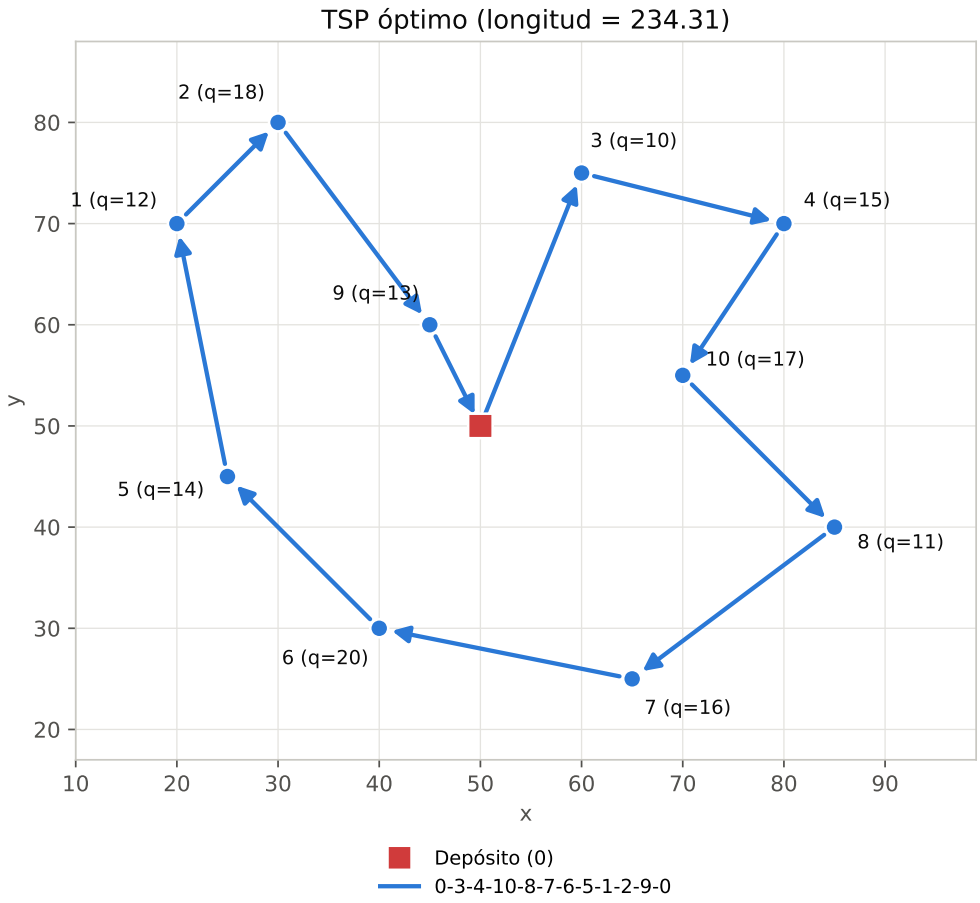


Figura 3: Recorrido TSP óptimo sobre los 11 nodos (cota inferior del CVRP).

Se cumple entonces $234.31 \leq Z^* = 318.99 \leq 339.25$: restringir la capacidad encarece el recorrido en 84.68 unidades (36.1 % sobre el TSP), y la heurística de ahorros queda 44.8 % por encima de la cota. Además, siguiendo a Prins (2004), un recorrido TSP puede verse como una *gran ruta* sin delimitadores que un procedimiento óptimo de partición (*split*) divide en rutas factibles. La Tabla 8 muestra que al partir la gran ruta del TSP se recupera exactamente la solución óptima del modelo exacto, mientras que la concatenación de las rutas de Clarke–Wright reproduce su propio costo. Esto no garantiza que el TSP conduzca siempre al óptimo, pero sí motiva la codificación de gran ruta que se utilizará en la etapa bioinspirada.

Tabla 8: Resultado del procedimiento *split* sobre dos grandes rutas.

Gran ruta	Z tras *split*	Rutas resultantes
3-4-10-8-7-6-5-1-2-9 (TSP)	318.99	0-3-4-10-0, 0-8-7-6-0, 0-5-1-2-0, 0-9-0
2-1-5-8-4-3-9-6-7-10 (Clarke–Wright)	339.25	0-2-1-5-0, 0-8-4-3-9-0, 0-6-7-0, 0-10-0

4. Definición del problema

Se busca determinar cuántos vehículos salen del depósito, qué clientes atiende cada uno y en qué orden, de forma que se atienda toda la demanda, ningún vehículo exceda su capacidad y la distancia total recorrida sea mínima. Se supone una flota homogénea y suficiente, demandas conocidas y no divisibles (cada cliente es atendido por un solo vehículo) y costos simétricos.

Conjuntos.

- $V = \{0, 1, \ldots, n\}$: nodos; el 0 es el depósito.
- $C = \{1, \ldots, n\} = V \setminus \{0\}$: clientes.
- $A = \{(i, j) : i, j \in V,\ i \neq j\}$: arcos.

Parámetros.

- c_{ij}: distancia del nodo i al nodo j (Tabla 4).
- q_i: demanda del cliente i (Tabla 3).
- Q: capacidad de cada vehículo ($Q = 50$).
- $K_{\text{mín}} = \lceil \sum_{i \in C} q_i / Q \rceil$: número mínimo de vehículos.

Variables de decisión.

- $x_{ij} \in \{0, 1\}$: vale 1 si algún vehículo viaja directamente de i a j; 0 en otro caso.
- $u_i \geq 0$: carga acumulada entregada por el vehículo al salir del cliente i.

5. Modelo matemático

Se utiliza una formulación de dos índices con restricciones tipo Miller–Tucker–Zemlin (MTZ) (Miller et al., 1960), adaptadas a capacidad:

$$\text{mín} \quad Z = \sum_{(i,j)\in A} c_{ij}\, x_{ij} \tag{1}$$

$$\text{s.a.} \quad \sum_{j\in V,\, j\neq i} x_{ij} = 1 \qquad \forall\, i \in C \tag{2}$$

$$\sum_{i\in V,\, i\neq j} x_{ij} = 1 \qquad \forall\, j \in C \tag{3}$$

$$\sum_{j\in C} x_{0j} = \sum_{i\in C} x_{i0} \geq K_{\text{mín}} \tag{4}$$

$$u_j \;\geq\; u_i + q_j - Q\,(1 - x_{ij}) + (Q - q_i - q_j)\, x_{ji} \qquad \forall\, i, j \in C,\; i \neq j \tag{5}$$

$$q_i \leq u_i \leq Q \qquad \forall\, i \in C \tag{6}$$

$$x_{ij} \in \{0, 1\} \qquad \forall\, (i, j) \in A \tag{7}$$

- (1) minimiza la distancia total recorrida.
- (2) y (3): de cada cliente sale y entra exactamente un arco, es decir, se visita una sola vez.
- (4): los vehículos que salen del depósito regresan a él, y salen al menos $K_{\text{mín}}$. Esta cota es redundante pero acelera al optimizador.
- (5): si el vehículo va de i a j, la carga acumulada en j incluye la de i más q_j; el término adicional $(Q - q_i - q_j)\, x_{ji}$ es el levantamiento (*lifting*) de Desrochers y Laporte (1991) adaptado al CVRP por Kara et al. (2004): si j precede inmediatamente a i, obliga a $u_j \geq u_i - q_i$, lo que ajusta la relajación lineal sin cortar soluciones factibles. Sigue impidiendo los subtours..
- (6): la carga acumulada nunca excede Q, lo que garantiza la capacidad de cada ruta.
- (7): naturaleza binaria de las variables de recorrido.

5.1. Diagrama del modelo

La Figura 4 relaciona los parámetros, las variables, las acciones de la operación y las restricciones.

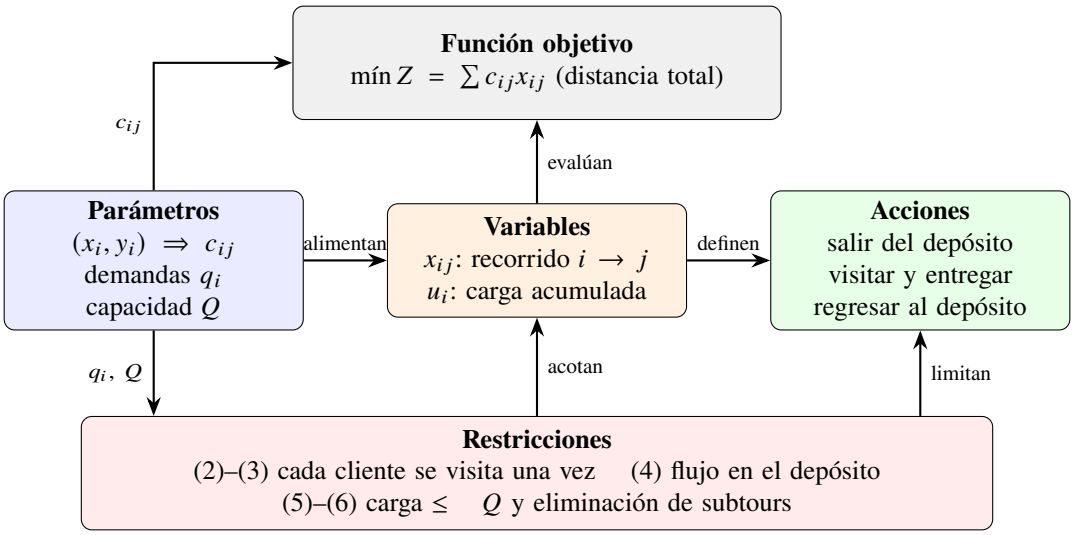


Figura 4: Relación entre parámetros, variables, acciones y restricciones del modelo.

6. Solución con el modelo exacto

El modelo (1)–(7) se implementó en Python con la librería PuLP y se resolvió con el optimizador CBC. Para la instancia CVRP_10, el modelo tiene 120 variables (110 binarias x_{ij} y 10 continuas u_i) y 112 restricciones, además de las cotas de las variables u_i.

CBC alcanzó el estatus *Optimal* en 1.61 segundos, con $Z^* = 318.99$ e igual cota inferior final, por lo que la optimalidad está demostrada. La relajación lineal del nodo raíz fue de 266.57, es decir, 16.4 % por debajo del óptimo. Por esa brecha, el optimizador exploró 3,806 nodos de ramificación aun con solo 10 clientes. Con la versión elevada de (5), la relajación lineal del nodo raíz sube de 266.57 a 276.64 (brecha de 13.3 % respecto al óptimo) y el óptimo no cambia ($Z^* = 318.99$), lo que confirma que el levantamiento no excluye soluciones factibles.

Tabla 9: Solución óptima obtenida con PuLP/CBC.

Ruta	Secuencia	Carga	Distancia	Espacio desperdiciado (%)
1	0-2-1-5-0	44	101.19	12.00
2	0-8-7-6-0	47	109.26	6.00
3	0-9-0	13	22.36	74.00
4	0-10-4-3-0	42	86.18	16.00
Total	4 rutas	146	318.99	27.00
Capacidad $Q = 50$; demanda total = 146; vehículos mínimos = $\lceil 146/50 \rceil = 3$				

La solución óptima también utiliza 4 rutas. Destaca la ruta 0-9-0, que atiende a un solo cliente con 74 % de espacio vacío. Esto ocurre porque el cliente 9 está a solo 11.18 unidades del depósito, de modo que su ida y vuelta (22.36) es más barata que desviar otra ruta para incluirlo. Esto confirma que, cuando el objetivo es la distancia, el número mínimo de vehículos no necesariamente es el óptimo. Si se incorporara un costo fijo

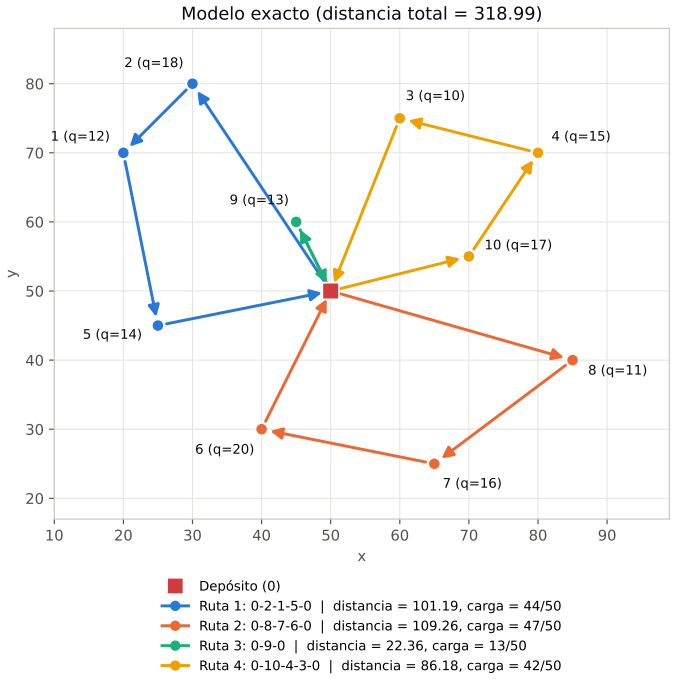


Figura 5: Rutas de la solución óptima.

por vehículo (Sección 1.2), el resultado podría cambiar.

6.1. Efecto del número de vehículos

Para cuantificar el efecto de la flota, se resolvió el problema forzando exactamente K rutas (Tabla 10). El mínimo teórico $K_{\text{mín}} = 3$ cuesta 339.16, es decir, 20.17 unidades más que el óptimo global con $K = 4$; a partir de $K = 5$ el costo vuelve a crecer. Si cada vehículo adicional tuviera un costo fijo F expresado en unidades de distancia, la función objetivo sería $Z + F \cdot K$ y usar 4 vehículos en lugar de 3 sería preferible mientras $F < 20.17$. Además, la solución de Clarke–Wright (339.25 con 4 rutas) resulta dominada por la mejor solución de 3 rutas (339.16) para cualquier $F \geq 0$.

Tabla 10: Mejor costo de la instancia CVRP_10 al fijar el número de rutas K.

K	Z	Rutas
3	339.16	0-5-6-7-0, 0-8-4-3-9-0, 0-1-2-10-0
4	318.99	0-9-0, 0-8-7-6-0, 0-3-4-10-0, 0-2-1-5-0
5	345.01	0-9-0, 0-7-8-0, 0-6-0, 0-3-4-10-0, 0-2-1-5-0
6	381.07	0-9-0, 0-7-8-0, 0-6-0, 0-5-0, 0-3-4-10-0, 0-1-2-0
7	418.22	0-10-0, 0-9-0, 0-7-0, 0-6-0, 0-5-0, 0-3-4-8-0, 0-1-2-0

7. Algoritmo de solución en Python

El procedimiento de la Sección 3 se programó en Python en dos fases. El código completo y su estructura se describen en el Anexo B.:

1. **Construcción:** heurística de ahorros de Clarke y Wright (1964) en su versión paralela, idéntica a la aplicada manualmente.
2. **Mejora:** búsqueda local con criterio de primera mejora, que se repite hasta que ningún movimiento reduce el costo. Usa tres vecindarios, todos respetando la capacidad Q:
   - *2-opt* intra-ruta: invierte un segmento de una ruta.
   - *Relocate* inter-ruta: mueve un cliente a otra ruta.
   - *Swap* inter-ruta: intercambia dos clientes de rutas distintas.

Como era de esperarse, la fase de construcción reprodujo exactamente la solución manual ($Z = 339.25$, Tabla 7). La fase de mejora no encontró ningún movimiento de mejora, por lo que la solución final es la misma (Tabla 11).

Tabla 11: Solución del algoritmo Clarke–Wright + búsqueda local.

Ruta	Secuencia	Carga	Distancia	Espacio desperdiciado (%)
1	0-2-1-5-0	44	101.19	12.00
2	0-8-4-3-9-0	49	119.82	2.00
3	0-6-7-0	36	77.01	28.00
4	0-10-0	17	41.23	66.00
Total	4 rutas	146	339.25	27.00
Capacidad $Q = 50$; demanda total = 146; vehículos mínimos = $\lceil 146/50 \rceil = 3$				

Este resultado indica que la solución de Clarke–Wright es un *óptimo local* respecto a los tres vecindarios. Las rutas 1 y 2 operan casi a capacidad completa (44 y 49 unidades), lo que hace infactible la mayoría de los movimientos. Los movimientos factibles empeoran el costo. Por ejemplo, mover el cliente 8 a la ruta 0-6-7-0 genera la ruta 0-6-7-8-0 del óptimo, pero aumenta Z en aproximadamente 1.5 unidades, por lo que la búsqueda local lo rechaza. Llegar al óptimo requiere aceptar movimientos que empeoran temporalmente la solución o combinar varios movimientos a la vez, que es justamente lo que hacen las metaheurísticas bioinspiradas revisadas en la Sección 2.

8. Comparación de resultados

La calidad de cada método se midió con el GAP porcentual respecto al óptimo, y el tiempo con su diferencia porcentual respecto al del modelo exacto:

$$\mathrm{GAP}_{\mathrm{costo}} = \frac{Z_{\mathrm{método}} - Z^*}{Z^*} \times 100, \qquad \Delta_{\mathrm{tiempo}} = \frac{t_{\mathrm{método}} - t^*}{t^*} \times 100.$$

El tiempo de los métodos heurísticos se reporta como la mediana de 100 ejecuciones, ya que una sola ejecución dura menos de un milisegundo y es sensible al ruido del sistema.

Tabla 12: Comparación entre el modelo exacto y el algoritmo.

Método	Z	N.º rutas	Desperdicio global (%)	Tiempo (s)	GAP costo (%)	Dif. tiempo (%)
Modelo exacto (PuLP/CBC)	318.99	4	27.00	1.614595	0.00	0.0000
Clarke-Wright	339.25	4	27.00	0.000093	6.35	-99.9943
Clarke-Wright + búsqueda local	339.25	4	27.00	0.000156	6.35	-99.9903

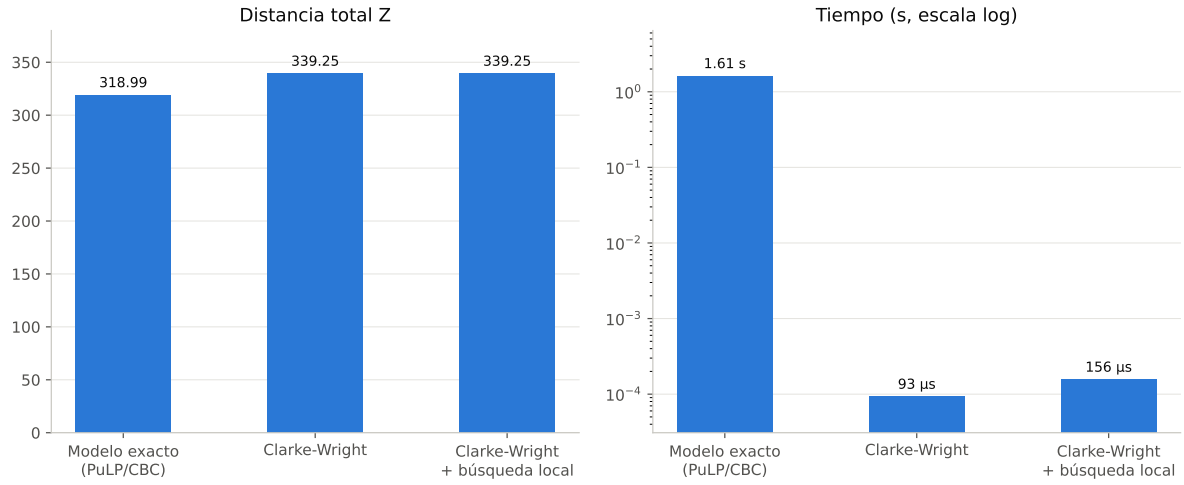


Figura 6: Distancia total (izquierda) y tiempo de cómputo en escala logarítmica (derecha) por método.

El algoritmo obtiene una solución 6.35 % más cara que el óptimo (20.26 unidades de distancia adicionales), pero en un tiempo 99.99 % menor: el modelo exacto tardó 1.61 s, mientras que Clarke–Wright tardó 9.3×10^{-5} s, unas 17,000 veces más rápido. La búsqueda local agrega tiempo (de 9.3×10^{-5} a 1.56×10^{-4} s) sin mejorar el costo en esta instancia.

Consideraciones sobre la medición del tiempo. El tiempo del modelo exacto corresponde al tiempo total de `solve` de PuLP, que incluye la escritura del modelo, el arranque del proceso externo de CBC y la lectura de la solución (Mitchell et al., 2011); el tiempo de los heurísticos es el de una función en memoria. Por ello, la razón de 17,000 debe leerse como un orden de magnitud y no como una comparación exacta de eficiencia algorítmica. Para reducir el ruido, el tiempo del modelo exacto se reporta como la mediana de R ejecuciones.

Ambas soluciones usan 4 rutas y tienen el mismo desperdicio global (27 %), pero distribuyen la carga de forma distinta. La diferencia de costo se explica por la asignación de los clientes 7, 9 y 10. El óptimo agrupa 8-7-6 y 10-4-3, clientes cercanos entre sí, y deja al 9 solo. La heurística, en cambio, fijó desde las primeras iteraciones la ruta 0-3-4-8-0, lo que forzó a dejar al 10 aislado.

9. Conclusiones

- Se construyó la matriz de costos de la instancia CVRP_10 y se resolvió manualmente con la heurística de ahorros, obteniendo 4 rutas y una distancia total de 339.25.
- El modelo de programación entera mixta con restricciones MTZ, resuelto con PuLP/CBC, obtuvo el óptimo demostrado $Z^* = 318.99$, también con 4 rutas, en 1.61 s.
- El algoritmo programado tiene un GAP de 6.35 % y es unas 17,000 veces más rápido que el modelo exacto. La búsqueda local no logró mejorarlo porque la solución es un óptimo local de vecindarios simples.
- Minimizar la distancia no implica minimizar el número de vehículos: con 3 rutas la demanda cabe, pero el óptimo en distancia usa 4.
- Aunque para 10 clientes el modelo exacto es viable, la débil relajación lineal (16.4 % de brecha) y los 3,806 nodos explorados anticipan un crecimiento acelerado del tiempo en instancias mayores. Esto, junto con el estancamiento de la búsqueda local, justifica desarrollar en las siguientes etapas un algoritmo bioinspirado capaz de escapar de óptimos locales.
- El TSP óptimo (234.31) es una cota inferior del CVRP: el óptimo (318.99) está 36.1 % por encima y la heurística de ahorros 44.8 %. Al partir la gran ruta del TSP con el procedimiento *split* se recupera el óptimo de esta instancia.
- Los factores a mejorar identificados en el caso (distancia, número de vehículos, uso de la capacidad y emisiones) pueden integrarse con una función objetivo ponderada $Z + F \cdot K$ (Sección 6) o tratarse como objetivos separados con un método multiobjetivo como NSGA-II (Deb et al., 2002); para la siguiente etapa se propone una codificación de gran ruta con *split* (Prins, 2004), ya validada en la Sección 3.6.

Referencias

Ai, T. J., & Kachitvichyanukul, V. (2009). Particle Swarm Optimization and Two Solution Representations for Solving the Capacitated Vehicle Routing Problem. *Computers & Industrial Engineering*, *56*(1), 380-387.

Baker, B. M., & Ayechew, M. A. (2003). A Genetic Algorithm for the Vehicle Routing Problem. *Computers & Operations Research*, *30*(5), 787-800.

Bektaş, T., & Laporte, G. (2011). The Pollution-Routing Problem. *Transportation Research Part B: Methodological*, *45*(8), 1232-1250.

Blum, C., & Roli, A. (2003). Metaheuristics in Combinatorial Optimization: Overview and Conceptual Comparison. *ACM Computing Surveys*, *35*(3), 268-308.

Bullnheimer, B., Hartl, R. F., & Strauss, C. (1999). An Improved Ant System Algorithm for the Vehicle Routing Problem. *Annals of Operations Research*, *89*, 319-328.

Clarke, G., & Wright, J. W. (1964). Scheduling of Vehicles from a Central Depot to a Number of Delivery Points. *Operations Research*, *12*(4), 568-581.

Crainic, T. G., Ricciardi, N., & Storchi, G. (2009). Models for Evaluating and Planning City Logistics Systems. *Transportation Science*, *43*(4), 432-454.

Dantzig, G. B., & Ramser, J. H. (1959). The Truck Dispatching Problem. *Management Science*, *6*(1), 80-91.

Deb, K., Pratap, A., Agarwal, S., & Meyarivan, T. (2002). A Fast and Elitist Multiobjective Genetic Algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, *6*(2), 182-197.

Desrochers, M., & Laporte, G. (1991). Improvements and Extensions to the Miller–Tucker–Zemlin Subtour Elimination Constraints. *Operations Research Letters*, *10*(1), 27-36.

Dijkstra, E. W. (1959). A Note on Two Problems in Connexion with Graphs. *Numerische Mathematik*, *1*, 269-271.

Fukasawa, R., Longo, H., Lysgaard, J., Poggi de Aragão, M., Reis, M., Uchoa, E., & Werneck, R. F. (2006). Robust Branch-and-Cut-and-Price for the Capacitated Vehicle Routing Problem. *Mathematical Programming*, *106*(3), 491-511.

Gendreau, M., Laporte, G., & Séguin, R. (1996). Stochastic Vehicle Routing. *European Journal of Operational Research*, *88*(1), 3-12.

Held, M., & Karp, R. M. (1962). A Dynamic Programming Approach to Sequencing Problems. *Journal of the Society for Industrial and Applied Mathematics*, *10*(1), 196-210.

Kara, I., Laporte, G., & Bektaş, T. (2004). A Note on the Lifted Miller–Tucker–Zemlin Subtour Elimination Constraints for the Capacitated Vehicle Routing Problem. *European Journal of Operational Research*, *158*(3), 793-795.

Laporte, G. (2009). Fifty Years of Vehicle Routing. *Transportation Science*, *43*(4), 408-416.

Lenstra, J. K., & Rinnooy Kan, A. H. G. (1981). Complexity of Vehicle Routing and Scheduling Problems. *Networks*, *11*(2), 221-227.

Miller, C. E., Tucker, A. W., & Zemlin, R. A. (1960). Integer Programming Formulation of Traveling Salesman Problems. *Journal of the ACM*, *7*(4), 326-329.

Mitchell, S., O'Sullivan, M., & Dunning, I. (2011). PuLP: A Linear Programming Toolkit for Python [Documentación en https://coin-or.github.io/pulp/].

Murray, C. C., & Chu, A. G. (2015). The Flying Sidekick Traveling Salesman Problem. *Transportation Research Part C*, *54*, 86-109.

Prins, C. (2004). A Simple and Effective Evolutionary Algorithm for the Vehicle Routing Problem. *Computers & Operations Research*, *31*(12), 1985-2002.

Prodhon, C., & Prins, C. (2014). A Survey of Recent Research on Location-Routing Problems. *European Journal of Operational Research*, *238*(1), 1-17.

Reimann, M., Doerner, K., & Hartl, R. F. (2004). D-Ants: Savings Based Ants Divide and Conquer the Vehicle Routing Problem. *Computers & Operations Research*, *31*(4), 563-591.

Schneider, M., Stenger, A., & Goeke, D. (2014). The Electric Vehicle-Routing Problem with Time Windows and Recharging Stations. *Transportation Science*, *48*(4), 500-520.

Solomon, M. M. (1987). Algorithms for the Vehicle Routing and Scheduling Problems with Time Window Constraints. *Operations Research*, *35*(2), 254-265.

Toth, P., & Vigo, D. (2014). *Vehicle Routing: Problems, Methods, and Applications* (2.ª ed.). SIAM.

Vidal, T. (2022). Hybrid Genetic Search for the CVRP: Open-Source Implementation and SWAP* Neighborhood. *Computers & Operations Research*, *140*, 105643.

Vidal, T., Crainic, T. G., Gendreau, M., Lahrichi, N., & Rei, W. (2012). A Hybrid Genetic Algorithm for Multidepot and Periodic Vehicle Routing Problems. *Operations Research*, *60*(3), 611-624.

A. Registro completo de la heurística de ahorros

Tabla 13: Evaluación de los 45 pares de clientes.

Iter.	(i, j)	s_{ij}	¿Fusión?	Motivo	Ruta resultante
1	(1, 2)	57.97	Sí	carga 30/50	0-1-2-0
2	(3, 4)	42.37	Sí	carga 25/50	0-3-4-0
3	(4, 8)	42.04	Sí	carga 36/50	0-3-4-8-0
4	(7, 8)	40.56	No	capacidad excedida (16+36>50)	–
5	(4, 10)	38.64	No	nodo interior (4)	–
6	(1, 5)	36.06	Sí	carga 44/50	0-2-1-5-0
7	(8, 10)	35.80	No	capacidad excedida (36+17>50)	–
8	(2, 3)	32.57	No	capacidad excedida (44+36>50)	–
9	(5, 6)	26.64	No	capacidad excedida (44+20>50)	–
10	(2, 5)	26.20	No	misma ruta	–
11	(6, 7)	26.02	Sí	carga 36/50	0-6-7-0
12	(3, 10)	25.18	No	capacidad excedida (36+17>50)	–
13	(1, 3)	22.67	No	nodo interior (1)	–
14	(2, 9)	22.24	No	capacidad excedida (44+13>50)	–
15	(2, 4)	21.12	No	nodo interior (4)	–
16	(3, 8)	20.31	No	misma ruta	–
17	(1, 9)	20.31	No	nodo interior (1)	–
18	(7, 10)	19.36	No	capacidad excedida (36+17>50)	–
19	(4, 7)	17.78	No	nodo interior (4)	–
20	(3, 9)	16.89	Sí	carga 49/50	0-8-4-3-9-0
21	(1, 6)	13.69	No	nodo interior (1)	–
22	(6, 8)	12.66	No	capacidad excedida (36+49>50)	–
23	(1, 4)	12.11	No	nodo interior (1, 4)	–
24	(5, 9)	11.68	No	capacidad excedida (44+49>50)	–
25	(4, 9)	10.84	No	misma ruta	–
26	(5, 7)	9.93	No	capacidad excedida (44+36>50)	–
27	(2, 10)	9.50	No	capacidad excedida (44+17>50)	–
28	(2, 6)	7.43	No	capacidad excedida (44+36>50)	–
29	(3, 5)	6.32	No	nodo interior (3)	–
30	(9, 10)	6.30	No	capacidad excedida (49+17>50)	–
31	(3, 7)	5.83	No	nodo interior (3)	–
32	(1, 10)	4.47	No	nodo interior (1)	–
33	(2, 8)	4.45	No	capacidad excedida (44+49>50)	–
34	(6, 10)	3.92	No	capacidad excedida (36+17>50)	–
35	(6, 9)	3.13	No	capacidad excedida (36+49>50)	–
36	(8, 9)	2.86	No	misma ruta	–
37	(4, 6)	1.85	No	nodo interior (4)	–
38	(5, 8)	1.69	No	capacidad excedida (44+49>50)	–
39	(1, 7)	1.57	No	nodo interior (1)	–
40	(4, 5)	1.14	No	nodo interior (4)	–
41	(1, 8)	0.87	No	nodo interior (1)	–
42	(3, 6)	0.04	No	nodo interior (3)	–
43	(7, 9)	0.02	No	capacidad excedida (36+49>50)	–
44	(2, 7)	0.02	No	capacidad excedida (44+36>50)	–
45	(5, 10)	0.01	No	capacidad excedida (44+17>50)	–

B. Repositorio de código y estructura del proyecto

Todo el código utilizado en este entregable (lectura de la instancia, matriz de costos, heurística de ahorros, búsqueda local, modelo exacto, cota TSP, comparación y gráficas) está disponible públicamente en el repositorio del equipo:

https://github.com/Sebastian-aleman/SP_Algoritmos-matematicos-bioinspirados/tree/main/SP%20bioinspirados/cvrp/src

La ruta `SP bioinspirados/cvrp/` contiene el proyecto de esta etapa. Está organizado separando los datos de entrada, el código reutilizable (módulos en `src/`), los scripts que ejecutan cada etapa y las salidas generadas, de modo que cualquier tabla o figura del reporte puede reproducirse volviendo a correr el script correspondiente.

B.1. Estructura de carpetas

```
SP bioinspirados/
|-- cvrp/
|   |-- datos/            instancia CVRP_10 (formato TSPLIB)
|   |-- src/              modulos reutilizables
|   |   |-- instancia.py
|   |   |-- matriz_costos.py
|   |   |-- clarke_wright.py
|   |   |-- busqueda_local.py
|   |   |-- algoritmo.py
|   |   |-- modelo_exacto.py
|   |   |-- tsp.py
|   |   |-- reporte.py
|   |   |-- comparacion.py
|   |   ‘-- graficas.py
|   |-- resultados/       tablas .tex, .csv, .json y log de CBC
|   |-- figuras/          graficas en .pdf para el reporte
|   |-- main_paso1.py     scripts de ejecucion
|   |-- main_paso3.py
|   |-- main_tsp.py
|   ‘-- requirements.txt  dependencias de Python
|-- Entrega1 (3).pdf     reporte entregado
‘-- .gitignore
```

La carpeta `datos/` guarda la instancia `CVRP_10` en formato TSPLIB, con las coordenadas, las demandas y la capacidad $Q = 50$. Las carpetas `resultados/` y `figuras/` se llenan automáticamente al ejecutar los scripts: la primera contiene las tablas en LaTeX (por ejemplo, `matriz_costos.tex`, `ahorros.tex`,

iteraciones_cw.tex, comparacion.tex), los respaldos en .csv y .json de las soluciones y el registro de CBC (cbc_modelo_exacto.log); la segunda contiene las gráficas de nodos, rutas y comparación en formato PDF. Así, las tablas y figuras que aparecen en este documento provienen directamente de esos archivos y no fueron capturadas a mano.

B.2. Módulos del directorio src/

La Tabla 14 resume la responsabilidad de cada módulo. Se mantuvo una función por tarea y los módulos no dependen de los scripts de ejecución, lo que permite reutilizarlos en las siguientes etapas (por ejemplo, el algoritmo bioinspirado podrá usar instancia.py, matriz_costos.py, reporte.py y tsp.py sin modificarlos).

Tabla 14: Módulos del directorio src/.

Módulo	Función
instancia.py	Lee una instancia CVRP en formato TSPLIB (EUC_2D), valida el archivo y re-indexa los nodos desde 0 para que el depósito sea el nodo 0.
matriz_costos.py	Calcula la matriz de distancias euclidianas sin redondear y la exporta a .csv (precisión completa) y a .tex (2 decimales).
clarke_wright.py	Calcula los ahorros s_{ij}, ejecuta la versión paralela de la heurística de ahorros y guarda el registro de cada fusión evaluada (Tablas 4, 5 y 10).
busqueda_local.py	Búsqueda local con primera mejora: 2-opt intra-ruta, *relocate* y *swap* inter-ruta, respetando la capacidad.
algoritmo.py	Integra la construcción y la mejora (resolver_clarke_wright, resolver_cvrp) y mide el tiempo como la mediana de varias ejecuciones.
modelo_exacto.py	Construye y resuelve con PuLP/CBC el modelo de dos índices con restricciones MTZ de carga (límite de 600 s), reconstruye las rutas y extrae del log la cota inferior y la relajación lineal.
tsp.py	TSP exacto con programación dinámica de Held y Karp y procedimiento *split* para dividir una gran ruta en rutas factibles.
reporte.py	Evalúa una solución (carga, distancia y porcentaje de espacio desperdiciado por ruta) y la exporta a .tex y .json.
comparacion.py	Calcula el GAP de costo, la diferencia porcentual de tiempo y el desperdicio global para comparar los métodos contra el óptimo.
graficas.py	Genera con matplotlib las figuras de nodos, rutas, comparación de métodos y recorrido TSP.

B.3. Scripts de ejecución y reproducción

Cada script es independiente y escribe sus resultados en resultados/ y figuras/: main_paso1.py lee la instancia, construye la matriz de costos, aplica la heurística de ahorros y genera la solución manual (Secciones 3.1 a 3.5 y Anexo A); main_paso3.py resuelve el modelo exacto, ejecuta Clarke–Wright con y sin búsqueda local, mide los tiempos (mediana de 100 ejecuciones para los heurísticos) y arma la comparación (Secciones 6 a 8); y main_tsp.py calcula la cota inferior TSP y la partición de la gran ruta. Para reproducir los resultados, desde la carpeta cvrp/ basta ejecutar:

```
pip install -r requirements.txt
python main_paso1.py
python main_paso3.py
python main_tsp.py
```

Los tiempos de cómputo dependen del equipo en que se ejecute el código, por lo que pueden diferir de los reportados en la Tabla 9, mientras que las distancias y rutas son deterministas.